
Automatic Detection of Personally Identifiable Information in Educational Texts

Allan Li

2025403473

Department of Computer Science
Tsinghua University

lpjl25@mails.tsinghua.edu.cn

En Yi Hou

2025403491

Department of Computer Science
Tsinghua University

hey25@mails.tsinghua.edu.cn

Alenna Tang

2025403493

Institute for Interdisciplinary Information Sciences
Tsinghua University

tjy25@mails.tsinghua.edu.cn

Abstract

1 Detecting personally identifiable information (PII) in text is critical for privacy but
2 difficult. In this project, we developed a machine learning model to automatically
3 identify PII tokens using the DeBERTa architecture. To prevent our model from
4 memorizing specific documents, we implemented a document grouped cross val-
5 idation strategy. We trained six different model variants with different settings,
6 such as testing with different model architectures and adjusting the weights of dif-
7 ferent PII entities during training process. We then combined these models using
8 different ensemble techniques. Our results showed that a simple averaging ensem-
9 ble and a single model with adjusted loss weights performed the best, achieving
10 an F beta score of approximately 0.95. Surprisingly, more complex methods like
11 meta classifier to combine predictions actually performed worse.

1 Problem

Large educational datasets collected from online learning platforms, educational technology tools, and academic research projects are increasingly used to study student behavior and improve teaching practices. However, these datasets frequently contain personally identifiable information (PII), such as student names, email addresses, usernames, and identification numbers. Releasing data that includes PII poses serious privacy risks and can violate legal and ethical frameworks such as the Family Educational Rights and Privacy Act (FERPA) [24], preventing datasets from being shared publicly.

The task addressed in this work is *token-level PII detection* in student essays. Given text that has been tokenized, the goal is to assign each token a BIO label corresponding to one of seven PII categories, or to the non-PII class (O) [10]. These labels can then be used to automatically identify and redact sensitive spans prior to dataset release. This specific formulation follows the 2024 Learning Agency Lab challenge focused on educational data privacy [23].

This task presents several challenges. First, PII tokens are relatively rare compared to non-PII tokens, leading to severe class imbalance, which can bias model predictions toward the majority class [8]. Second, student writing is noisy and informal, often containing typos, partial identifiers, or unconventional formatting. Finally, many PII instances require contextual disambiguation. For example, a token such as “Washington” may refer to a location, a cited author, or a student name depending on context. Models must therefore detect sensitive identifiers accurately without over-redacting non-sensitive content.

2 Motivation

Manual PII redaction remains the most reliable approach for protecting student privacy, but it is slow, expensive, and not scalable to large datasets [15]. As dataset sizes grow, manual review also becomes vulnerable to reviewer fatigue, increasing the likelihood of inconsistent annotations or missed identifiers [14]. Even a single unredacted instance of PII can block dataset release or cause privacy harm [13].

Automated PII detection offers a way to:

- Scale privacy-safe dataset release,
- Reduce the cost of human review,
- Enable broader access to educational data for research.

The motivation for automated PII detection also extends beyond education. Many organizations invest significant resources to comply with privacy regulations such as the General Data Protection Regulation (GDPR) and the California Consumer Privacy Act (CCPA) [15, 19]. In digital payment systems, transaction logs and customer interactions contain sensitive PII that must be sanitized to prevent cyber-attacks and data leakage [16], while in healthcare, automated de-identification is essential for enabling analytics and machine learning on electronic health records without compromising patient privacy [17]. In addition, the widespread adoption of Large Language Models (LLMs) increases the importance of accurate PII detection. Models trained on insufficiently sanitized data risk memorizing and later reproducing sensitive user information, a phenomenon often referred to as training data leakage [1].

Because the consequences of missing PII are substantially more severe than those of over-redaction, we prioritize high recall while maintaining sufficient precision to keep sanitized text usable for downstream applications. This tradeoff directly motivates both our modeling choices and our evaluation strategy.

3 Methods & Techniques

We define the task of Personally Identifiable Information (PII) detection as a supervised token classification problem (Sequence Labeling). Let $X = \{x_1, x_2, \dots, x_n\}$ represent an input sequence of

59 n tokens derived from a student essay. Our objective is to learn a mapping function $f_\theta : X \rightarrow Y$,
 60 where $Y = \{y_1, y_2, \dots, y_n\}$ corresponds to the sequence of predicted labels [9].

61 The label space is defined as $\mathcal{L} = \{l_O\} \cup \mathcal{P}$, where l_O represents the non-sensitive ‘‘Outside’’ token
 62 and $\mathcal{P}$ represents the set of 7 PII categories (e.g., NAME_STUDENT, PHONE_NUM) annotated in BIO
 63 format.

For each token x_i , the model computes a conditional probability distribution over the label space $\mathcal{L}$
 given the bidirectional context of the entire sequence X :

$$P(y_i = c | X) = \text{softmax}(W \cdot \mathbf{h}_i + b)_c$$

64 where $\mathbf{h}_i$ is the contextualized embedding of token x_i and $c \in \mathcal{L}$. Our optimization goal is to
 65 maximize the log-likelihood of the correct label sequence, effectively minimizing the cross-entropy
 66 loss between the predicted distribution and the ground truth.

67 3.1 Model Architecture: The Backbone

68 To approximate the function f_θ with high fidelity, our system employs the DeBERTa-v3 architecture
 69 [6]. We selected this over standard BERT [3] and RoBERTa [12] due to its superior capability
 70 in handling spatially dependent token relations, which is necessary for resolving ambiguous PII
 71 contexts (e.g., distinguishing a cited author from a student signature).

72 3.1.1 DeBERTa Primer: Disentangled Attention

73 Traditional transformer architectures like BERT encode positional information by summing content
 74 and position embeddings ($E_{\text{input}} = E_{\text{content}} + E_{\text{position}}$), a method that entangles semantics with
 75 syntax and limits the model’s ability to learn precise relative positioning [6].

In contrast, DeBERTa utilizes a **disentangled attention mechanism**, representing each token with
 two distinct vectors: content (H) and relative position (P). Attention scores are computed using
 disentangled matrices [6]:

$$A_{i,j} = H_i^T H_j + H_i^T P_{i|j} + P_{i|j}^T H_j$$

76 This separation allows the model to distinguish between semantic similarity and syntactic proximity,
 77 which is important for identifying PII boundaries (e.g., distinguishing a name in a citation from a
 78 name in a signature) [20]. Furthermore, DeBERTa integrates an **Enhanced Mask Decoder (EMD)**
 79 that injects absolute position embeddings immediately prior to the softmax layer, recovering global
 80 context often lost in relative encoding schemes [6].

81 3.2 System Overview

82 The proposed solution utilizes a hierarchical **Stacked Generalization (Stacking)** architecture de-
 83 signed to maximize detection recall while maintaining high precision. As illustrated in Figure 1,
 84 the system pipeline is divided into two distinct processing stages: the *Variant Inferencing Layer*
 85 (Level-0) and the *Metaclassifier Layer* (Level-1).

86 The workflow proceeds as follows:

- 87 1. **Input Processing:** Raw educational texts are ingested and tokenized using the DeBERTa-
 88 v3 tokenizer.
- 89 2. **Variant Inferencing Layer (Level-0):** The tokenized input is fed in parallel into a collec-
 90 tion of fine-tuned DeBERTa-v3 variants. To ensure ensemble diversity, these variants are
 91 trained with differing hyperparameters and loss functions:
 - 92 • **Baseline DeBERTa:** Standard fine-tuning.
 - 93 • **Weighting Strategies:** Variants utilizing static `o_weight` (penalizing the ‘‘O’’ class)
 94 and `dynamic` weighting to handle class imbalance.
 - 95 • **Regularization Strategies:** Variants utilizing Multi-Sample Dropout (`md`) to improve
 96 generalization.
 - 97 • **Hybrid Approaches:** Combinations such as `md_dynamic` and `md_o_weight`.

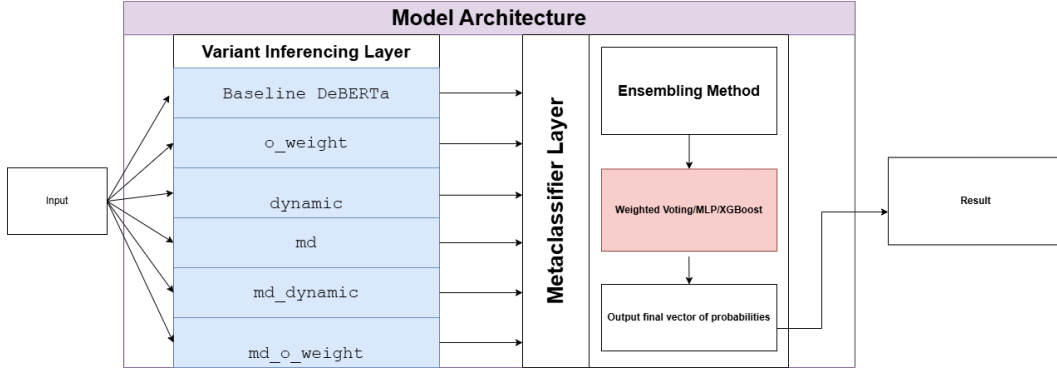


Figure 1: **High-Level Model Architecture.** The input text is processed in parallel by diverse DeBERTa variants. Their probabilistic outputs are aggregated and fed into a Metaclassifier (Weighted Voting, MLP, or XGBoost) to generate the final PII predictions.

3. **Feature Aggregation:** Each variant outputs a vector of token-level probabilities (logits). These vectors are concatenated to form a dense feature representation of the text, capturing multiple “views” of the potential PII.
4. **Metaclassifier Layer (Level-1):** The aggregated logits serve as the input for the Metaclassifier. We experiment with three distinct ensembling methods (**Weighted Voting**, **Multilayer Perceptron (MLP)**, and **XGBoost**) to learn the optimal combination of the base models’ strengths.
5. **Result:** The Metaclassifier computes the final vector of probabilities, which is then decoded into BIO tags (e.g., B-NAME_STUDENT) to produce the final annotated document.

3.2.1 Ensemble Diversity & Hyperparameters

To maximize ensemble utility via error independence [25], we trained eight DeBERTa variants with diverse sensitivities to class imbalance. We employed Focal Loss [11] to address the extreme sparsity of PII tokens and varying dropout rates to alternate between conservative and aggressive prediction strategies (Table 1).

Table 1: Hyperparameter Configuration of the 6 DeBERTa Variants. We explore two architectural modes (Standard vs. Multi-Sample Dropout) across three distinct loss weighting strategies to encourage ensemble diversity.

Variant Name	Architecture	Weighting Strategy	Key Hyperparameters
baseline	Standard DeBERTa	Uniform	None
o_weight	Standard DeBERTa	Static O-Penalty	$w_O = 0.05$
dynamic	Standard DeBERTa	Dynamic Frequency	Batch-Dependent
md	Multi-Sample Dropout	Uniform	$k = 5, p = 0.2$
md_o_weight	Multi-Sample Dropout	Static O-Penalty	$k = 5, p = 0.2, w_O = 0.05$
md_dynamic	Multi-Sample Dropout	Dynamic Frequency	$k = 5, p = 0.2$

3.3 Training Strategy & Ensembling

To synthesize the diverse inductive biases of the six DeBERTa variants, we employed a **Stacked Generalization (Stacking)** framework [25].

3.3.1 Stacked Generalization Protocol

We partitioned the dataset into an 80/20 train-validation split to facilitate a two-level training process:

- 118 1. **Level-0 (Base Models):** The eight DeBERTa variants were fine-tuned exclusively on the
119 80% training partition.
- 120 2. **Level-1 (Meta-Classifer):** To train the meta-classifier, we generated prediction vectors
121 (logits) using the Level-0 models on the held-out 20% validation set.

122 Per Wolpert [25], training the meta-classifier on predictions from unseen data is critical to preventing
123 data leakage. This forces the ensemble to learn from the *generalization errors* of the base models
124 rather than their training set memorization.

125 3.3.2 Ensemble Aggregation Methods

126 We evaluated three distinct architectures for the Level-1 meta-classifier:

- 127 • **Weighted Averaging:** A baseline approach using static coefficients assigned based on
128 individual validation performance.
- 129 • **Multilayer Perceptron (MLP):** A learnable neural network head designed to capture non-
130 linear relationships between variant confidence scores.
- 131 • **XGBoost:** A gradient-boosted decision tree classifier. We selected XGBoost for its proven
132 superiority in handling tabular data (the logit vectors) and its ability to learn complex deci-
133 sion boundaries based on probability thresholds [2].

134 3.3.3 Training Implementation and Cost-Sensitive Optimization

135 Section 3.5 shows an extreme label imbalance, with `O` (non-PII) tokens accounting for more than
136 99.9% of the corpus. To prevent training from collapsing into always predicting `O`, we used cost-
137 sensitive learning [4] by modifying the token-level cross-entropy loss:

$$\mathcal{L} = - \sum_{c=1}^C w_c y_c \log(\hat{y}_c).$$

138 **Static down-weighting of `O`.** In the `o_weight` setting, we applied a fixed weight of $w_o = 0.05$
139 and set $w_c = 1.0$ for all PII labels. This reduces the gradient contribution of the majority class so
140 that updates reflect errors on rare PII tokens rather than background text [11].

141 **Dynamic class reweighting.** In addition to static reweighting, we implemented a `dynamic`
142 scheme that recomputes class weights from the empirical label frequencies observed in the current
143 training split. Concretely, we estimate a frequency f_c for each PII label from the training data and
144 assign larger weights to rarer labels, while keeping `O` down-weighted. This shifts learning pressure
145 toward under-represented entity types and reduces systematic false negatives on rare PII categories.

146 **Cross-validation protocol.** We trained and evaluated using deterministic K -fold cross-validation
147 ($K = 5$). Rather than splitting at the sentence level, we assigned each full document to a fold using
148 document ID modulo ($ID \bmod K$), ensuring that an essay and its context appear in only one fold.
149 This prevents document-level leakage and avoids inflated validation performance from partial essay
150 memorization.

151 3.4 Evaluation Metrics

152 3.4.1 Primary Metric: Micro- F_β Score (F_5)

153 To align with the competition’s privacy-centric objectives, we evaluated our system using the **Micro-**
154 **Averaged F_β Score** with $\beta = 5$ [9]. Unlike the standard F_1 score which balances precision and
155 recall equally, the F_5 score introduces a significant bias towards recall.

Mathematically, the F_β score is defined as the weighted harmonic mean of precision (P) and recall (R):

$$F_\beta = (1 + \beta^2) \cdot \frac{P \cdot R}{(\beta^2 \cdot P) + R}$$

By setting $\beta = 5$, the formula weights recall 25 times more heavily than precision in the harmonic mean calculation [18].

Justification for $\beta = 5$: In the context of Personally Identifiable Information (PII) redaction, the cost of errors is asymmetric.

- **False Negative (Missed PII):** Failing to detect a student’s name results in a privacy leak, which is a critical failure.
- **False Positive (Over-redaction):** Incorrectly flagging a generic word (e.g., “University”) merely results in a slight loss of readability or a low-cost manual review.

Therefore, our optimization objective prioritizes minimizing leakage (maximizing recall) even at the expense of precision.

3.4.2 Secondary Metrics: Precision vs. Recall

While F_5 serves as the primary optimization target, we continuously monitored **Precision** and **Recall** independently to diagnose model behavior.

- **Recall (Sensitivity):** Used to detect “blind spots” in the model, particularly for rare classes like PHONE_NUM where training data is sparse.
- **Precision (Positive Predictive Value):** Used to analyze the “noise” level of our synthetic data. A precipitous drop in precision typically indicated that our augmentation pipeline was generating unrealistic artifacts that confused the model.

3.5 Data Engineering

3.5.1 Dataset Composition

The dataset consists of approximately 22,000 essays with a severe class imbalance. As illustrated in Figure 2, the “O” (Outside) tag accounts for 99.95% of all tokens. Positive PII samples are extremely sparse; for example, B-PHONE_NUM appears in negligible quantities (≈ 6 instances), biasing the model toward false negatives.

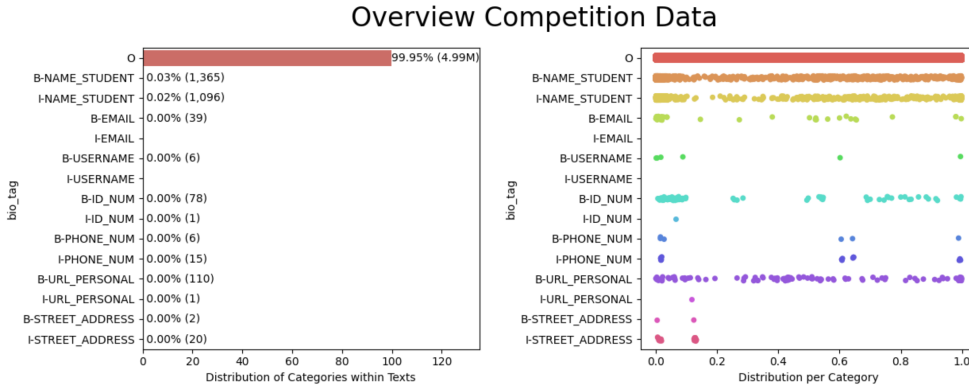


Figure 2: **Class Imbalance Analysis.** 99.95% of tokens are non-PII, creating a significant challenge for learning rare entities.

3.5.2 Synthetic Data Generation Pipeline

To address this sparsity, we implemented a synthetic data pipeline (Figure 3) utilizing Mistral-7B and Faker to generate high-density PII samples with 100% label accuracy.

The process involves three stages:

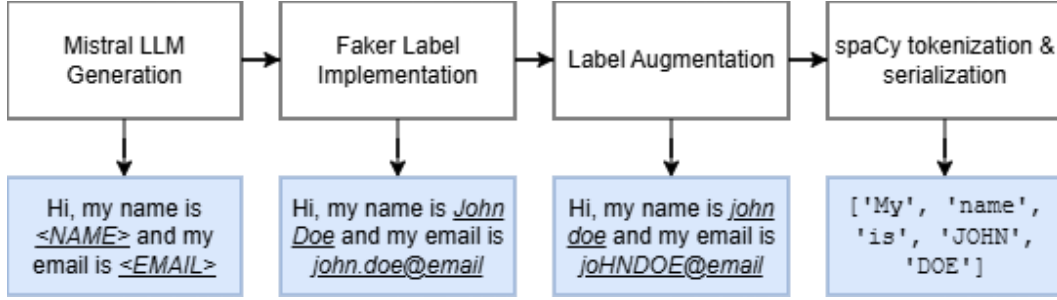


Figure 3: **Generation Pipeline.** Mistral-7B generates templates with placeholders, which are populated by Faker and tokenized via spaCy.

Table 2: Pooled token-level performance by variant (micro F_5 , precision, recall).

Variant	F_5	Precision	Recall
baseline	0.8827	0.7436	0.8894
md	0.8872	0.7664	0.8928
dynamic	0.9453	0.4615	0.9867
md_dynamic	0.9392	0.4587	0.9802
md_o_weight	0.9443	0.5525	0.9719
o_weight	0.9485	0.5654	0.9749

1. **Context Generation (LLM):** We prompt Mistral-7B to generate academic-style text templates with specific placeholders (e.g., <NAME_STUDENT>), ensuring semantic coherence.
2. **Entity Injection:** Placeholders are mapped to Faker providers, guaranteeing strict alignment between the text and labels.
3. **Noise Injection:** To prevent overfitting to clean patterns, we apply stochastic corruption based on observed student error patterns:
 - **Morphological Corruption** ($p = 0.3$): Random character noise.
 - **Format Variation** ($p = 1.0$): Alternating separators for IDs/phones (e.g., “123-456” vs “123.456”).
 - **Case Jittering** ($p = 0.6$): Random case modifications (e.g., “john doe”).
 - **Whitespace Artifacts** ($p = 0.1$): Random whitespace injection.

4 Results and Analysis

4.1 Variants strengths and weaknesses

Table 2 reports the pooled scores, and Figure 4 shows the same results as precision and recall operating points. The variants fall into two groups. The uniformly weighted models (baseline, md) behave conservatively: they keep precision high but miss more PII, so recall stays lower. The reweighted models (o_weight, dynamic, and their md variants) do the opposite, as they flag PII much more aggressively, which raises recall substantially, but it also increases over-redaction and reduces precision.

Why we considered o_weight and dynamic. The baseline model is fairly cautious. It gets high precision, which means it usually does not label ordinary words as PII. The problem is recall: when the model is unsure, it often falls back to $\emptyset$. That is an easy habit to learn under extreme class imbalance, because most tokens in the data are $\emptyset$. During training, the loss is dominated by getting $\emptyset$ right, so the model can improve the objective just by predicting $\emptyset$ more often. Under a recall-heavy metric like F_5 , this conservative behavior is not what we want, because a missed identifier is a privacy leak.

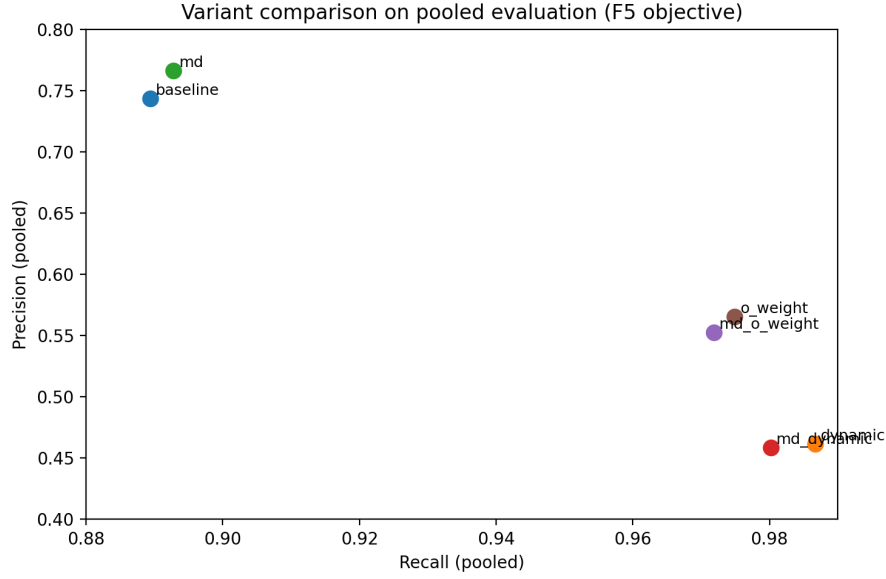


Figure 4: Precision-recall scatter on pooled evaluation. Loss reweighting shifts the operating point toward higher recall at the expense of precision.

We introduced `o_weight` and `dynamic` to push the model away from this default. Both methods reduce how much `O` contributes to the loss and increase the relative importance of PII labels. In effect, they make missed PII more expensive and make occasional over-tagging less costly.

This design matches our evaluation goal. We optimize micro- F_5 , which weights recall much more than precision, so the score drops sharply when the model misses PII. In other words, a false negative hurts far more than a false positive under F_5 . By penalizing missed PII more heavily during training, `o_weight` and `dynamic` push the model toward higher recall and reduce the exact error that F_5 treats as most costly.

Observed effect and the `o_weight` vs. `dynamic` trade-off. Reweighting does what we wanted: the recall-oriented variants jump to very high recall (about 0.97–0.99), and the F_5 score rises with it. The real decision is not whether reweighting helps, but how much extra over-redaction we are willing to accept in exchange for catching more PII.

`dynamic` is the most aggressive setting. It reaches the highest recall (0.9867), but precision falls sharply to 0.4615. In practice, this means the model prefers to tag when it is unsure, so it flags many borderline tokens as PII. `o_weight` is less extreme: it still achieves very high recall (0.9749) while keeping noticeably better precision (0.5654). Under a recall-heavy metric like F_5 , `o_weight` ends up as the best single-variant compromise because it captures most of the recall gain without triggering the same level of widespread over-redaction.

Observed Multi-dropout effect and trade-off. Multi-sample dropout (`md`) slightly improves pooled precision under uniform weighting (`md` vs `baseline`), which is consistent with variance reduction and smoother probability estimates observed in modern transformer architectures [7]. Its impact is more informative under recall-oriented weighting. Comparing `md_o_weight` to `o_weight` shows that multi-dropout can recover some precision while keeping recall high, suggesting that a portion of the false positives come from borderline tokens where stochastic averaging reduces overconfident spikes. However, multi-dropout does not fundamentally change the error regime: reweighting still dominates the operating point.

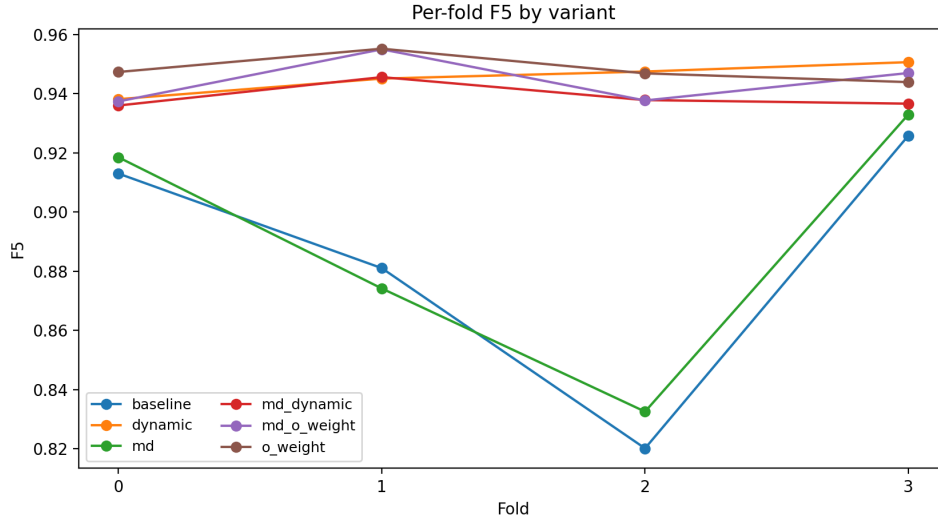


Figure 5: Per-fold F_5 by variant (folds 0, 1, 2 and 3)

4.2 Cross-fold stability

We decided to use k-fold cross validation because data is scarce. The real dataset contains only about 6.6k essays, and we generated roughly 20k additional synthetic samples. If we held out a large fixed validation set, we would lose a meaningful portion of the limited data for training.

At the same time, generating even more synthetic data would introduce many issues: it increases cost, and more importantly, synthetic pool cannot fully reflect real-world student writing. Therefore, such a decision risks shifting the training distribution toward synthetic patterns. If synthetic text starts to dominate, the model can become excellent at recognizing the style of our generator while getting worse at the messy, inconsistent edge cases that appear in real essays. In other words, more synthetic data can indeed provide more training data, but it also reduces realism, which could hurt our trained model’s real-world performance [21].

Each fold trains on most of the data and validates on a different slice, so every example contributes to training in some runs and to evaluation in others. This gives us two benefits at once: better use of limited data for learning, and a more reliable estimate of generalization than a single split that can be “lucky” or “unlucky” depending on which rare identifiers land in the validation set. By averaging performance across multiple folds, we significantly reduce the variance of our evaluation, providing a more robust measure of how the model will perform on unseen real-world data [5].

The fold-wise results also provide a diagnostic signal. If a variant performs well only on one fold, it suggests sensitivity to the particular mix of entities or writing styles in that split, which is a warning sign for deployment. Figure 5 illustrates this effect. Uniform-weighted variants (*baseline*, *md*) show noticeably larger fold-to-fold variation, while recall-oriented variants remain tightly clustered. Under severe imbalance, this difference is expected: when a fold contains fewer positive examples of PII, a uniform-loss model receives less updates that discourage predicting $\emptyset$ on true PII, so recall can drop sharply. Loss reweighting counteracts that failure mode by making rare positives matter even when they appear sparsely, which stabilizes F_5 across splits.

Therefore, k-fold is crucial to the model’s performance, as it reduces the chance that a single split accidentally under-represents a rare label and causes us to misjudge a variant. With only one held-out set, a model can look strong simply because the validation slice happens to contain fewer hard cases. K-fold averages over multiple such slices and makes that kind of evaluation error less likely.

From a privacy perspective, we are responsible to think from the worst-case scenario. A model that performs well on average but occasionally drops recall on a particular slice means that it can leak

Table 3: Error trade-off ($\circ$ vs any PII). FP/10k $\circ$ = number of non-PII tokens incorrectly flagged as PII per 10,000 true $\circ$ tokens (over-redaction rate). FN rate (PII) = percentage of true PII tokens incorrectly predicted as $\circ$ (miss rate).

Variant	FP ($\circ \rightarrow$ PII)	FN (PII $\rightarrow$ $\circ$)	FP/10k $\circ$	FN rate (PII)
baseline	783	267	1.77	10.15%
md	702	268	1.59	10.19%
dynamic	3008	14	6.81	0.53%
md_dynamic	3025	33	6.84	1.25%
md_o_weight	2052	55	4.64	2.09%
o_weight	1948	42	4.41	1.60%

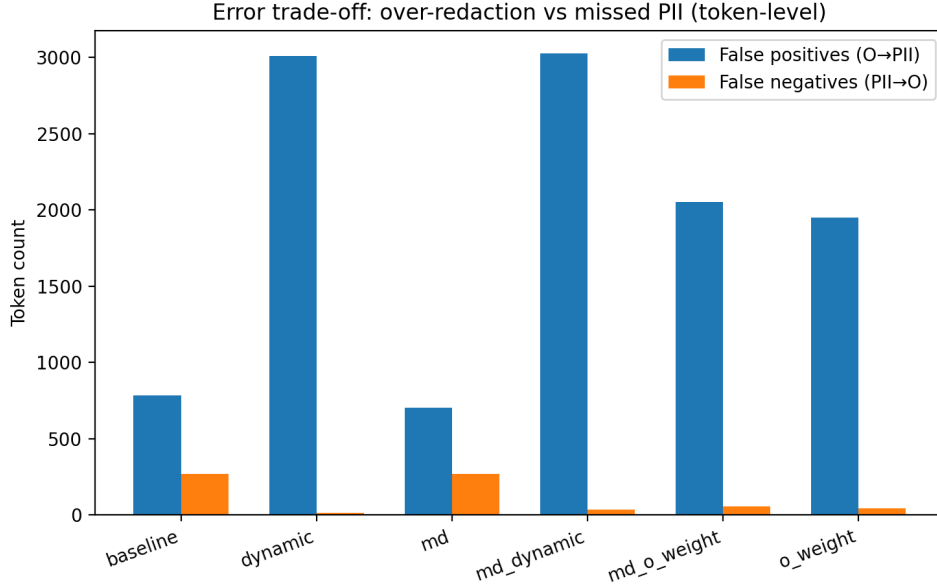


Figure 6: Error trade-off (token-level). Recall-oriented weighting reduces false negatives (missed PII) but increases false positives (over-redaction).

identifiers in some scenarios. Fold-wise evaluation makes those drops visible, so we can address such issue by choosing variants that behave accurately and consistently.

4.3 Error asymmetry: Missed PII versus over-redaction

In the context of PII redaction, the most critical failure is false negatives (true PII predicted as $\circ$). Table 3 and Figure 6 quantify the two error types categorized by PII types.

Key observation: reweighting increases recall and false positives. Uniform-weighted models miss about 10% of PII tokens, which is large for a privacy task. Reweighting reduces this dramatically: `o_weight` drops false negatives to 1.6%, and `dynamic` drops them to 0.53%. This is the main reason the F_5 score jumps from about 0.88 to about 0.95 as shown in Table 2.

The cost is equally clear: `dynamic` produces roughly $4\times$ more false positives than baseline (3008 vs 783). The `dynamic`-weighted model behaves like a high-sensitivity detector that flags many borderline tokens as PII. Under F_5 , this trade is acceptable because recall dominates.

Why `dynamic` loses more precision than `o_weight`. The two weighting schemes push the model toward recall, but they push in different ways. With `dynamic`, the loss weights change with label frequency, so the model pays a large penalty when it misses a rare PII label and a comparatively

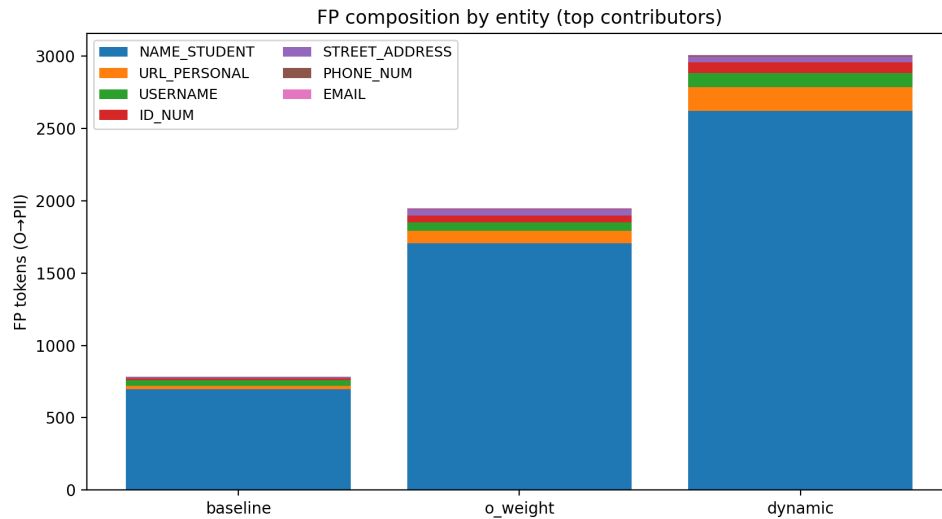


Figure 7: False-positive composition (O→PII) by entity for baseline, o_weight, and dynamic. Over-redaction concentrates heavily in NAME_STUDENT.

Table 4: False positives concentrate in NAME_STUDENT. Counts are derived from the provided confusion matrices (true O predicted as PII).

Variant	Total FP	FP as NAME_STUDENT	Share
baseline	783	697	89.0%
o_weight	1948	1705	87.5%
dynamic	3008	2621	87.1%

smaller penalty when it incorrectly tags an O token. In practice, this changes the model’s default behavior in borderline cases: when a token looks even slightly “name-like” or “ID-like,” the safer choice under the loss is to tag it rather than abstain, because an occasional false positive is cheaper than a missed PII token.

By contrast, o_weight applies one fixed down-weight to O across the board. This still discourages the trivial “predict O everywhere” solution, but it does not keep escalating the pressure on rare classes as aggressively as dynamic. As a result, the model remains more willing to leave genuinely ambiguous tokens as O. This difference shows up in Figure 6 where o_weight captures most of the recall improvement while avoiding the large precision drop observed with dynamic.

4.4 Where false positives come from: concentration in NAME_STUDENT

The drop in pooled precision is not evenly distributed across entity types. Instead, false positives concentrate overwhelmingly in a single class: NAME_STUDENT. Figure 7 and Table 4 show that roughly 87–92% of all false-positive tokens across variants are predicted as student names, even though several other entity types also exhibit recall variability.

Reconciling entity difficulty with overall impact. At first glance, this behavior may seem inconsistent with the entity-level recall heatmap in Figure 8, where entities such as ID_NUM and URL_PERSONAL appear to be more challenging for the model, especially under uniform weighting. However, difficulty and impact are not the same.

Identifiers such as ID_NUM, URL_PERSONAL, and PHONE_NUM occur relatively rarely in educational essays. Even when their recall is imperfect, the absolute number of affected tokens is small. As a result, errors on these entities contribute only marginally to the overall F_5 score and to the pooled precision statistics.

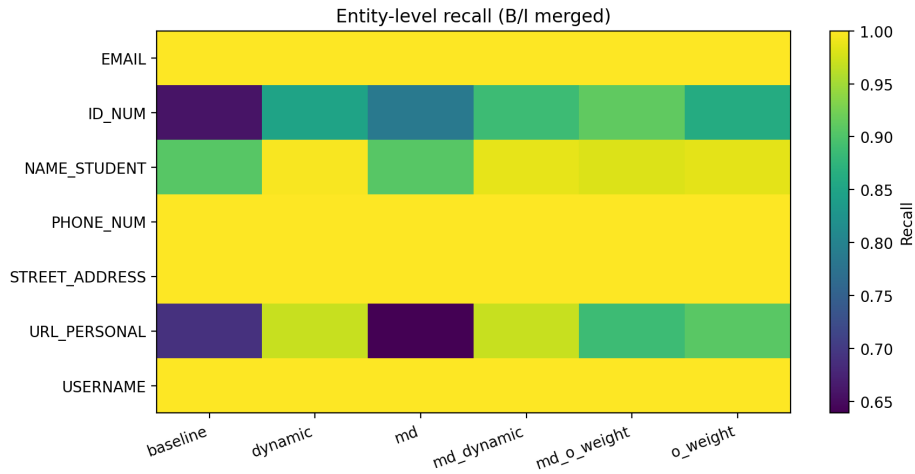


Figure 8: Entity-level recall with B/I merged. Recall-oriented training improves recall most for harder, sparser entities (e.g., ID_NUM, URL_PERSONAL).

NAME_STUDENT behaves very differently. Names appear far more frequently in educational text, both as true PII and in many non-sensitive contexts such as citations, acknowledgements, group work descriptions, and references to historical or fictional figures [22]. Consequently, even a modest increase in the false-positive rate for NAME_STUDENT produces a large absolute number of errors, which dominates the precision drop at the corpus level.

Why abundant training data does not solve the problem. Importantly, this failure mode is not caused by data scarcity. NAME_STUDENT is the most well-represented PII class in the training set, and the model sees far more examples of student names than of structured identifiers such as ID numbers or URLs. The issue is therefore not lack of supervision, but semantic ambiguity.

Names overlap heavily with ordinary language. Capitalized words, surnames, organization names, locations, and author citations often share surface-level features with student names. When recall-oriented weighting is applied, the model learns that misclassifying a true name as O is costly, while misclassifying a name-like token as PII is comparatively cheap. In ambiguous contexts, the model therefore defaults to tagging rather than abstaining.

Names overlap heavily with ordinary language. Capitalized words, surnames, organization names, locations, and author citations often share surface-level features with student names. When recall-oriented weighting is applied, the model learns that misclassifying a true name as O is costly, while misclassifying a name-like token as PII is comparatively cheap. In ambiguous contexts, the model therefore defaults to tagging rather than abstaining. This behavior is a direct consequence of cost-sensitive learning, where the optimal decision boundary is shifted toward the majority class to minimize the expected total cost of errors [4].

Implication for model improvement. These observations suggest that further gains are unlikely to come from simply increasing recall or adding more name examples. Instead, improving precision requires better disambiguation of name-like tokens, for example through richer contextual cues, stronger negative sampling for non-PII capitalized words, or post-processing constraints that suppress isolated name predictions without supporting context.

4.5 Entity-level recall: which classes benefit from reweighting

Figure 8 reports recall of different PII entities categorized by model variants. We observed that reweighting does not improve every class equally. The biggest gains concentrate in the entity types that the baseline model misses most often, especially ID_NUM and URL_PERSONAL, which are both sparse and highly variable in how they appear in student writing.

Table 5: Top false-negative entities (true PII predicted as $\circ$).

Variant	NAME_STUDENT	URL_PERSONAL	ID_NUM
baseline	218	30	19
o_weight	32	8	2
dynamic	9	2	3

Table 6: Combination strategies. Learned stackers raise precision but reduce recall enough to underperform under F_5 .

Method	F_5	Precision	Recall
variant_ensemble (final)	0.9482	0.5828	0.9726
mlp_stacker	0.8657	0.8358	0.8670
xgb_stacker	0.8477	0.9703	0.8434
fold_ensemble_mean (avg over variants)	0.9245	0.5913	0.9493

Table 5 makes the same point from a different angle by counting false negatives. Compared to baseline, both `o_weight` and `dynamic` dramatically reduce misses for `URL_PERSONAL` and `ID_NUM`, and they also cut missed `NAME_STUDENT` tokens by a large margin.

At first glance, `NAME_STUDENT` should be the easiest PII entity to be redacted because it appears frequently in the dataset so the model could learn its pattern accurately. However, the baseline results show the opposite: it still misses many name tokens (218 in Table 5). That gap suggests that the main difficulty is not only frequency, but ambiguity. Real essays contain many capitalized words that are not student names: place names, school names, author names in citations, or even the first word of a sentence. When the model faces a token like “Jordan” or “Washington”, it has to decide whether it is sensitive PII or just normal content, and the baseline often chooses $\circ$ to avoid false alarms.

Reweighting reduces these misses sharply by making the cost of abstaining on a true name higher. However, this improvement comes with a side effect that shows up elsewhere in the analysis: once the model becomes more willing to tag, it also over-tags name-like words, which increases false positives for `NAME_STUDENT`. In other words, reweighting helps the system meet the privacy objective (fewer leaked identifiers), but it shifts more of the workload into post-review by producing more aggressive redaction around ambiguous name-shaped tokens.

4.6 Final model: ensembling vs stacking

Table 6 compares combination strategies. The simple `variant_ensemble` achieves strong F_5 with a better precision–recall balance than the most aggressive single variant.

Why stacking underperforms despite higher model capacity. At first glance, the poor performance of the learned stackers is surprising. Meta-classifiers are generally expected to outperform simple averaging because they can exploit non-linear interactions among base model outputs. XGBoost, in particular, is widely regarded as a strong learner for tabular representations such as logit vectors, and its high capacity suggests that it should be able to identify subtle patterns that a fixed averaging scheme cannot capture.

The empirical results, however, reveal that this potential advantage does not translate into better performance in our setting. The main issue is not a lack of expressive power, but the conditions under which the meta-classifier is trained. Only 20% of the dataset is available for learning the combination rule. While this constraint is necessary for methodological correctness, it significantly limits the amount of data available to the meta-classifier, especially for learning reliable behavior on rare PII labels.

There is also a misalignment between how the stackers are trained and how the system is evaluated. Although the final evaluation metric is F_5 , which places a strong emphasis on recall, the

meta-classifiers are optimized using standard loss functions that implicitly reward overall accuracy. Under severe imbalance, this objective favors majority-class decisions and encourages conservative predictions. As a result, the stackers effectively reintroduce the dominance of the `O` class that loss reweighting was meant to mitigate at the base-model level.

By contrast, simple averaging avoids learning an explicit decision boundary altogether. Instead, it retains the recall-oriented behavior of the individual variants while tempering extreme confidence scores through aggregation. In a low-data, high-imbalance regime, this turns out to be advantageous. Rather than overfitting to sparse and biased Level-1 supervision, averaging provides a form of implicit regularization. In this context, the simplicity of the approach helps prevent the ensemble from collapsing back into a conservative solution dominated by the majority class.

4.7 Limitations supported by the results

The most prominent limitation is about over-redaction. While loss reweighting substantially improves recall and sharply reduces missed PII, it does so by increasing the number of false positives by roughly a factor of 2.5 to 4. These additional errors are not uniformly distributed across entity types. Instead, they are heavily concentrated in `NAME_STUDENT`, indicating that the model frequently assigns name labels to tokens that are only weakly suggestive of personal identifiers. This behavior suggests that further improvements are unlikely to come from pushing recall higher, and instead require better disambiguation of name-like tokens through richer contextual modeling and more informative negative examples.

A second limitation concerns the reliability of entity-level conclusions for rare labels. Some PII categories appear only a handful of times in the dataset, which makes their precision and recall estimates inherently unstable. Although the cross-fold analysis shows that overall F_5 performance is stable, this does not fully resolve uncertainty at a more microscopic level of individual PII entities.

Finally, the results expose a trade-off between robustness and inference efficiency. Techniques that improve stability and balance the precision–recall trade-off, such as multi-dropout inference and variant-level ensembling, incur additional computational cost at prediction time. Running multiple forward passes or multiple models increases latency and resource usage, which may be impractical in low-latency deployment settings. In such cases, a single recall-oriented model such as `o_weight` represents a more efficient compromise.

5 Conclusion

We studied the problem of token-level detection of personally identifiable information (PII) in student writing, a privacy-sensitive setting characterized by extreme class imbalance and contextual ambiguity. The goal was to reduce the risk of leaking sensitive identifiers while maintaining practical usability for downstream applications and dataset release.

Our results show that loss reweighting is essential under extreme imbalance, particularly static `o_weight` and dynamic weighting. Standard transformer models trained with uniform loss weighting behave conservatively under imbalance, achieving high precision but missing a non-trivial fraction of PII. Recall-oriented variants dramatically reduce missed PII compared to uniformly weighted baselines, but this improvement comes with a predictable increase in false positives. Under a recall-heavy objective such as the F_5 metric, this trade-off is appropriate. Because false negatives represent privacy failures while false positives primarily incur low-cost over-redaction, models that aggressively favor recall achieve substantially better alignment with the task objective, even when precision decreases.

The analysis further demonstrates that this trade-off is structured rather than random. False positives concentrate heavily in the `NAME_STUDENT` category, indicating that name-like tokens represent the dominant source of ambiguity. Multi-sample dropout improves stability and slightly recovers precision, but does not fundamentally alter the recall–precision operating point. Ensemble averaging across diverse variants provides the best overall balance, preserving high recall while smoothing the most aggressive predictions. In contrast, learned stackers favor precision and reintroduce majority-class bias, making them poorly aligned with a recall-heavy objective such as F_5 .

420 Over-redaction remains the primary failure mode, with false positives concentrated in ambiguous
421 name-like tokens, and more aggressive weighting strategies substantially increase annotation noise.
422 In addition, ensemble robustness improves performance but introduces higher inference-time com-
423 putational cost.

424 These findings suggest that future work should focus on improving name disambiguation without
425 sacrificing recall, for example through stronger contextual modeling, span-level objectives, or post-
426 processing constraints that limit spurious NAME_STUDENT predictions. More broadly, within the
427 scope of our experiments using a fixed transformer backbone, performance differences were driven
428 primarily by objective design and ensembling strategy rather than by changes in model capacity or
429 architecture.

References

- [1] Jaydeep Borkar. What can we learn from data leakage and unlearning for law? In *Proceedings of the ICML 2023 Workshop on Generative AI + Law (GenLaw)*, Honolulu, HI, USA, July 2023. URL <https://arxiv.org/abs/2307.10476>. arXiv preprint arXiv:2307.10476.
- [2] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. URL <https://arxiv.org/abs/1603.02754>.
- [3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, 2019. URL <https://aclanthology.org/N19-1423/>.
- [4] Charles Elkan. The foundations of cost-sensitive learning. *Proceedings of the Seventeenth International Conference on Artificial Intelligence: 4-10 August 2001; Seattle*, 1, 05 2001.
- [5] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Science & Business Media, New York, NY, second edition, 2009. See Chapter 7 on Model Assessment and Selection.
- [6] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: Decoding-enhanced bert with disentangled attention. In *International Conference on Learning Representations*, 2021. URL <https://arxiv.org/abs/2006.03654>.
- [7] Hiroshi Inoue. Multi-sample dropout for accelerated training and better generalization. *arXiv preprint arXiv:1905.09788*, 2019. URL <https://arxiv.org/abs/1905.09788>.
- [8] Justin M. Johnson and Taghi M. Khoshgoftaar. Survey on deep learning with class imbalance. *Journal of Big Data*, 6(1):1–54, 2019.
- [9] Kaggle. Pii data detection and removal from educational data: Overview. <https://www.kaggle.com/competitions/pii-detection-removal-from-educational-data/overview>, 2023. Accessed: November 11, 2025.
- [10] Guillaume Lample, Miguel Ballesteros, Sandeep Subramanian, Kazuya Kawakami, and Chris Dyer. Neural architectures for named entity recognition. In *Proceedings of the 2016 Conference on North American Chapter of the Association for Computational Linguistics*, pages 260–270, 2016.
- [11] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2980–2988, 2017. URL <https://arxiv.org/abs/1708.02002>.
- [12] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019. URL <https://arxiv.org/abs/1907.11692>.
- [13] Nomad Data. Privacy law compliance: Automating PII redaction in claim files. <https://www.nomad-data.com/doc-chat/privacy-law-compliance-automating-pii-redaction-in-claim-files-workers-compensat> 2025. Accessed: 2026-01-07.
- [14] Ayush Parti. Annotation fatigue: Why human data quality declines over time. <https://pareto.ai/blog-posts/annotation-fatigue-why-human-data-quality-declines-over-time>, February 2025. Accessed: 2025-11-11.
- [15] PwC. Privacy: A strategic reset. <https://www.pwc.com/us/en/services/consulting/cybersecurity-risk-regulatory/library/privacy-reset.html>, 2021. Accessed: November 11, 2025.
- [16] Kalyanasundharam Ramachandran. Securing PII data in payment transactions: Challenges and solutions. *ResearchGate*, June 2024. Technical Report.

- 483 [17] Narasimha Chaitanya Samineni. PII de-identification techniques for healthcare data ware-
484 houses. *ResearchGate*, December 2024. Technical Report.
- 485 [18] Towards Data Science. Understanding precision, recall
486 and f-beta score. [https://towardsdatascience.com/
487 understanding-precision-recall-and-f-beta-score-84d8a32c1e07,](https://towardsdatascience.com/understanding-precision-recall-and-f-beta-score-84d8a32c1e07)
488 2019. Accessed: 2026-01-07.
- 489 [19] Secureframe. Gdpr compliance costs and time savings. [https://secureframe.com/
490 hub/gdpr/cost-and-time-savings](https://secureframe.com/hub/gdpr/cost-and-time-savings), 2024. Accessed: November 11, 2025.
- 491 [20] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position rep-
492 resentations. In *Proceedings of the 2018 Conference of the North American Chapter of the*
493 *Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short*
494 *Papers)*, pages 464–468, 2018. URL <https://aclanthology.org/N18-2074/>.
- 495 [21] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Ander-
496 son. AI models collapse when trained on recursively generated content. *Nature*, 631:755–759,
497 2024. doi: 10.1038/s41586-024-07566-y.
- 498 [22] Shreya Singhal, Andres Felipe Zambrano, Maciej Pankiewicz, Xiner Liu, Chelsea Porter,
499 and Ryan S. Baker. De-identifying student personally identifying information with gpt-4.
500 In Benjamin PaaŸen and Carrie Demmans Epp, editors, *Proceedings of the 17th Inter-*
501 *national Conference on Educational Data Mining*, pages 559–565, Atlanta, Georgia, USA,
502 July 2024. International Educational Data Mining Society. ISBN 978-1-7336736-5-5. doi:
503 10.5281/zenodo.12729884.
- 504 [23] The Learning Agency Lab. PII data detection and removal from ed-
505 ucational data. [https://www.kaggle.com/competitions/
506 pii-detection-removal-from-educational-data](https://www.kaggle.com/competitions/pii-detection-removal-from-educational-data), 2024. Kaggle Compe-
507 tition.
- 508 [24] U.S. Department of Education. Family educational rights and privacy act (FERPA). [https :
509 //studentprivacy.ed.gov/node/548/](https://studentprivacy.ed.gov/node/548/), 1974. Accessed: 2026-01-07.
- 510 [25] David H Wolpert. Stacked generalization. *Neural networks*, 5(2):241–259, 1992.